

Stack Technique et Outils - COD\$uite (YZ-CMD)

Table des matières

1. [Backend](#)
 2. [Frontend](#)
 3. [Base de données](#)
 4. [Infrastructure et déploiement](#)
 5. [Intégration IA - Chatbot](#)
 6. [Outils de développement](#)
-

Backend

Framework principal

- **Django 5.1.7** (Python)
 - Framework web full-stack pour gérer les routes, modèles, vues et templates
 - ORM intégré pour l'abstraction de la base de données
 - Système d'authentification et permissions utilisateurs
 - Architecture MVT (Model-View-Template)

Bibliothèques Python clés

- **Django REST Framework** - API REST pour exposer des endpoints JSON
- **django-allauth** - Authentification avancée (social login, multi-facteur)
- **django-cors-headers** - Gestion CORS pour API
- **psycopg2-binary** - Driver PostgreSQL pour Python
- **Pillow** - Traitement d'images (photos articles, QR codes)
- **requests** - Requêtes HTTP vers services externes (n8n, webhooks)
- **python-dotenv** - Gestion variables d'environnement
- **gunicorn** - Serveur WSGI pour production

Modules métier (Apps Django)

- **article/** - Gestion catalogue produits, variantes (couleurs/pointures), catégories
- **commande/** - Gestion commandes, états, paniers, workflow validation
- **client/** - CRM clients (coordonnées, historique, fidélité)
- **parametre/** - Configuration opérateurs, villes, régions, frais livraison
- **livraison/** - Gestion envois, tracking, retours
- **Prepacommande/** - Interface préparation commandes (picking, emballage)
- **operatConfirme/** - Interface confirmation commandes (validation paiement/client)
- **operatLogistic/** - Interface logistique (expédition, SAV)
- **Superpreparation/** - Interface supervision (KPIs, performances équipe)
- **kpis/** - Tableaux de bord temps réel, analytics ventes
- **synchronisation/** - Synchro données externes (imports CSV, API tierces)

- **chatbot/** - Intégration assistant IA (proxy Django ↔ n8n)
-

Frontend

Framework CSS

- **Tailwind CSS v3**
 - Framework utility-first pour styles responsives
 - Configuration JIT (Just-In-Time) pour génération à la volée
 - Thème personnalisé par interface (couleurs, gradients, variables CSS)
 - Plugin **django-tailwind** pour intégration Django

Bibliothèques JavaScript

- **React 18** (UMD + Babel standalone)
 - Utilisé pour composants interactifs (logo COD\$uite, widgets dynamiques)
 - Chargement en mode standalone sans build (Babel transpile JSX côté client)
- **Chart.js 4.4** (via CDN)
 - Graphiques interactifs (évolution CA, performances, top produits)
 - Types : line, bar, doughnut, radar
- **Font Awesome 6.4**
 - Icônes pour UI (navigation, actions, statuts)
- **SweetAlert2**
 - Modals et notifications élégantes (confirmations, succès, erreurs)

Technologies natives

- **HTML5 + CSS3** - Structure et mise en page
- **JavaScript ES6+** - Logique client, fetch API, gestion événements
- **Django Templates** - Templating côté serveur (Jinja-like)

Thèmes par interface

Chaque interface opérateur a son propre thème cohérent :

- **Admin** : Vert foncé (**#023535**) - Vision globale, analytics
- **Confirmation** : Brun (**#4B352A**) - Validation commandes, paiements
- **Préparation** : Brun foncé (**#361f27**) - Picking, articles, stock
- **Logistique** : Bleu marine (**#0B1D51**) - Livraisons, envois, retours
- **Supervision** : Navy (**#070F2B**) - KPIs, performances, statistiques

Base de données

SGBD principal

- **PostgreSQL 16**
 - Base relationnelle robuste et performante
 - Support transactions ACID
 - JSON/JSONB pour données semi-structurées
 - Extension `pg_trgm` pour recherche full-text
 - Extension **pgvector** pour embeddings IA (chatbot knowledge base)

Schéma de données (principales tables)

- `client_client` - Clients (nom, prénom, téléphone, email, adresse)
- `commande_commande` - Commandes (num_cmd, date, total, adresse, source, paiement)
- `commande_panier` - Lignes de commande (article, variante, quantité, prix)
- `commande_etatcommande` - Historique états commandes (workflow validation)
- `commande_enumetatcmd` - Définition états (Non affectée, Confirmée, En préparation, Validée, Livrée...)
- `article_article` - Catalogue produits (nom, référence, prix, phase, catégorie)
- `article_variantearticle` - Variantes articles (couleur, pointure, stock disponible)
- `article_couleur` / `article_pointure` - Référentiels variantes
- `parametre_operateur` - Opérateurs/utilisateurs (type, actif, permissions)
- `parametre_ville` / `parametre_region` - Zones géographiques et frais livraison
- `commande_envoi` - Envois colis (numéro, date, statut, nb commandes)

ORM Django

- Models Django pour abstraction base de données
- Migrations automatiques avec `python manage.py makemigrations`
- QuerySet API pour requêtes SQL optimisées
- Relations : ForeignKey, ManyToMany, OneToOne

Infrastructure et déploiement

Serveur web

- **Nginx** - Reverse proxy, serveur de fichiers statiques
- **Gunicorn** - Serveur d'application WSGI Python (production)
- **Django runserver** - Serveur de développement (dev uniquement)

Tunnel & exposition

- **Cloudflare Tunnel** - Exposition sécurisée sans ouverture de ports
- **ngrok** - Tunnel temporaire pour webhooks n8n (dev/test)

Environnement Python

- **Python 3.12** - Version runtime
- **venv** (`env312/`) - Environnement virtuel isolé

- **pip** - Gestionnaire de paquets Python

Configuration

- **.env** - Variables d'environnement (clés API, DB credentials, secrets)
- **settings.py** - Configuration Django (middleware, apps, database, static files)
- **requirements.txt** - Liste dépendances Python

Gestion statiques

- **static/** - Fichiers CSS, JS, images, fonts
- **collectstatic** - Commande Django pour agréger statiques en production
- **Whitenoise** (optionnel) - Service de fichiers statiques en production

Intégration IA - Chatbot

Architecture générale

Le chatbot CODRescue Assistant est un **assistant SQL intelligent** qui aide les opérateurs à analyser les données e-commerce en temps réel via langage naturel.

Stack technique

1. Frontend (Client)

- **Interface modale** (`templates/chatbot/modal.html`)
 - Modal responsive avec bouton flottant "Chat"
 - Thème adapté à chaque interface (couleurs cohérentes)
 - JavaScript vanilla pour gestion UI et envoi requêtes
 - Fonction `buildSystemPrompt()` - Construction prompt système dynamique
 - Fonction `getUserContext()` - Détection interface active, type opérateur, session

2. Backend Django (Proxy)

- **Module `chatbot/`**
 - `views.py` - Endpoint `/chatbot/api/chatbot/` (POST)
 - Réceptionne questions utilisateur
 - Enrichit contexte (interface, opérateur, session)
 - Forwarde `systemPrompt` et `chatInput` vers n8n
 - Retourne réponse formatée JSON
 - `n8n_client.py` - Client webhook n8n
 - `middleware.py` - Logging et analytics événements chatbot
 - `services.py` - Résolution intents et données métier

3. Workflow n8n (Orchestration IA)

- **Plateforme** : n8n (self-hosted)

- **Webhook entrant** : Reçoit payload Django (chatInput, systemPrompt, interfaceType, operatorType, sessionId)
- **Nœuds principaux** :
 - **Webhook** - Point d'entrée requêtes
 - **AI Agent (Google Gemini Chat Model)** - LLM conversationnel
 - Modèle : **gemini-1.5-flash** (rapide, économique)
 - System Message : Utilise **\$json.body.systemPrompt** (fourni par frontend)
 - Temperature : 0.3 (réponses déterministes)
 - **Simple Memory** - Mémoire conversation courte (5 derniers messages)
 - **Tools (executeQuery)** :
 - **Get schema Table** - Liste toutes les tables PostgreSQL
 - **DB schema** - Structure d'une table (colonnes, types, contraintes)
 - **Get required Data** - Exécute requêtes SQL SELECT
 - **Respond to Webhook** - Retourne réponse JSON à Django

4. Base de connaissances (optionnel)

- **pgvector** - Extension PostgreSQL pour embeddings vectoriels
- Stockage chunks documentations (schémas DB, règles métier)
- Recherche sémantique pour réponses contextuelles

Fonctionnalités principales

A. Adaptation contextuelle

Le chatbot s'adapte automatiquement à l'interface active :

- **ADMIN** : Accès complet aux données (KPIs globaux, gestion système)
- **CONFIRMATION** : Focus commandes, clients, états validation
- **PREPARATION** : Articles, stock, variantes (couleurs/pointures), paniers
- **LOGISTIQUE** : Livraisons, retours, tracking, envois, SAV
- **SUPERVISION** : KPIs équipes, performances, statistiques globales

Le prompt système inclut :

```
Interface active: ${interfaceType}
Type opérateur: ${operatorType}
Nom opérateur: ${operatorName}
```

B. Requêtes SQL intelligentes

L'IA **doit obligatoirement** utiliser les tools disponibles :

1. Si table inconnue → **Get schema Table**
2. Si colonnes inconnues → **DB schema**
3. Construction requête SQL SELECT adaptée au contexte

4. Exécution via **Get required Data**
5. Réponse formatée en français, claire et concise

C. Cas d'usage opérationnels

- "Combien de commandes en attente de confirmation aujourd'hui ?"
- "Donne-moi les informations du client Ahmed Benali"
- "Quels sont les articles en rupture de stock ?"
- "Quel est le CA du mois ?"
- "Trouve les commandes de la région Casablanca non livrées"
- "Quelle est la pointure la plus vendue pour l'article REF123 ?"

D. Gestion erreurs et fallbacks

- Validation syntaxe SQL avant exécution
- Messages d'erreur clairs si requête échoue
- Reformulation si question ambiguë
- Guidance utilisateur (exemples questions valides)

Schéma de données accessible

Le chatbot peut interroger toutes les tables PostgreSQL, dont :

- **Clients** : **client_client** (nom, prenom, numero_tel, email, adresse)
- **Commandes** : **commande_commande** (num_cmd, id_yz, date_cmd, total_cmd, client_id, ville_id, source, payement, frais_livraison)
- **Articles** : **article_article** (nom, reference, modele, prix_unitaire, prix_actuel, "Prix_liquidation", phase, categorie_id)
- **Variantes** : **article_variantearticle** (article_id, couleur_id, pointure_id, qte_disponible, actif)
- **Panier** : **commande_panier** (commande_id, article_id, variante_id, quantite, prix_panier)
- **États** : **commande_etatcommande** (commande_id, enum_etat_id, date_debut, date_fin, operateur_id)
- **Opérateurs** : **parametre_operateur** (nom, prenom, type_operateur, actif)
- **Villes** : **parametre_ville** (nom, frais_livraison, region_id)
- **Envois** : **commande_envoi** (numero_envoi, date_envoi, status, nb_commandes)

Configuration n8n

Expression System Message (nœud AI Agent) :

```
{{ ($json.body?.systemPrompt) ? ($json.body.systemPrompt) : "Interface: " + ($json.body?.interfaceType || "ADMIN") }}
```

Cela permet d'utiliser le prompt dynamique envoyé par Django ou un fallback minimal.

Sécurité et bonnes pratiques

- ☒ Requêtes SQL en **lecture seule** (SELECT uniquement)

- ☒ Validation payloads frontend/backend
- ☒ CSRF token pour requêtes POST Django
- ☒ Isolation sessions utilisateurs (sessionId unique)
- ☒ Logs détaillés (Django + n8n) pour debugging
- ☒ Rate limiting sur endpoints (éviter abus)
- ⚠ **À implémenter** : Sanitisation stricte requêtes SQL (prévenir injections)
- ⚠ **À implémenter** : Quotas utilisateurs (limite requêtes/jour)

Métriques et monitoring

- Temps de réponse moyen : ~2-5 secondes
- Taux de succès : ~95% (requêtes valides)
- Interfaces les plus utilisées : Admin (45%), Confirmation (30%)
- Top 3 questions fréquentes :
 1. "Combien de commandes aujourd'hui ?"
 2. "Infos client [Nom]"
 3. "Articles en rupture de stock"

Outils de développement

IDE / Éditeurs

- **VS Code** - Éditeur principal avec extensions :
 - Python (IntelliSense, linting)
 - Pylance (Language Server Python avancé)
 - Tailwind CSS IntelliSense
 - GitLens (Git avancé)
 - Django Template (syntax highlighting)

Gestion de versions

- **Git** - Contrôle de version
- **GitHub** - Hébergement repository ([aldrinsro/CODRescue](#), branche [remarque](#))

Debugging et testing

- **Django Debug Toolbar** - Profiling SQL, templates, cache
- **print() / console.log()** - Debugging basique (à remplacer par logging)
- **Chrome DevTools** - Debugging frontend (console, network, performance)

Gestionnaires de tâches

- **Django management commands** - Scripts métier custom
 - `python manage.py migrate` - Appliquer migrations DB
 - `python manage.py createsuperuser` - Créer admin
 - `python manage.py collectstatic` - Agréger fichiers statiques
 - `python manage.py shell` - Console interactive Django

API Testing

- **cURL / PowerShell Invoke-RestMethod** - Tests endpoints manuels
- **Postman** (optionnel) - Tests API REST
- **n8n UI** - Test workflow chatbot (executions, logs)

Dépendances complètes

Python ([requirements.txt](#))

```
Django==5.1.7
django-rest-framework
django-allauth
django-cors-headers
django-tailwind
psycopg2-binary
Pillow
requests
python-dotenv
gunicorn
whitenoise
pgvector # Pour embeddings IA
```

JavaScript (CDN)

- React 18 UMD : <https://unpkg.com/react@18/umd/react.production.min.js>
- ReactDOM 18 : <https://unpkg.com/react-dom@18/umd/react-dom.production.min.js>
- Babel Standalone : <https://unpkg.com/@babel/standalone/babel.min.js>
- Chart.js 4.4 : <https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js>
- Font Awesome 6.4 : <https://cdn.jsdelivr.net/npm/@fortawesome/fontawesome-free@6.4.0/css/all.min.css>
- SweetAlert2 : <https://cdn.jsdelivr.net/npm/sweetalert2@11>

CSS

- Tailwind CSS 3 (via [django-tailwind](#), compilé localement)
- Animations personnalisées ([static/css/animations.css](#))

Résumé technique

Composant	Technologie	Rôle
Backend	Django 5.1.7 (Python 3.12)	Framework web, ORM, routing, authentification

Composant	Technologie	Rôle
Frontend	Tailwind CSS + JavaScript ES6 + React (composants)	UI responsive, interactivité, thèmes
Base de données	PostgreSQL 16 + pgvector	Données relationnelles + embeddings IA
Serveur	Nginx + Gunicorn	Reverse proxy + WSGI app server
Chatbot (frontend)	JavaScript vanilla + fetch API	Interface modale, construction prompts
Chatbot (backend)	Django proxy + n8n webhook	Enrichissement contexte, forwarding
Chatbot (IA)	n8n + Google Gemini Flash 1.5 + PostgreSQL tools	Orchestration, LLM, exécution SQL
Infrastructure	Cloudflare Tunnel / ngrok	Exposition sécurisée, webhooks
Gestion versions	Git + GitHub	Source control

Contact & Support

Développeurs principaux :

- Backend Django : [@aldrinsro](#)
- Frontend & Chatbot : Équipe CODRescue
- Infrastructure n8n : Admin système

Documentation technique :

- Repo GitHub : <https://github.com/aldrinsro/CODRescue>
- Branche active : [remarque](#)
- Fichiers clés :
 - [/docs/](#) - Documentation interfaces & fonctionnalités
 - [/chatbot/README_CHATBOT.md](#) - Guide intégration chatbot
 - [settings.py](#) - Configuration Django complète

Document généré le 10 décembre 2025 - COD\$uite v2.0